# Chapter 8 — Flexbox

Normal flow (Chapter 7) stacks blocks and wraps text, nothing more. Side-by-side boxes, equal heights or vertical centring used to need floats, tables or negative-margin hacks. **Flexbox** (the CSS Flexible Box Layout module, a W3C Candidate Recommendation supported everywhere for about a decade) solves the **one-dimensional** case: lay items out in a **row or a column**, share out spare space, and align them — navbars, toolbars, button groups, cards, form rows, centring. For two-dimensional layouts (rows *and* columns) use Grid (Chapter 9).

**In this chapter:** 8.1 Container and items · 8.2 The two axes, direction and `order` · 8.3 `flex-grow` / `flex-shrink` / `flex-basis` (calculated by hand, verified in Chrome) · 8.4 The `flex` shorthand and `min-width: auto` · 8.5 Alignment and auto margins · 8.6 Wrapping and `gap` · 8.7 Common patterns · 8.8 Pitfalls, debugging and interview prep.

Every example is a page in `code/` with its screenshot in `img/`: containers have a **dashed purple border**, items a **solid blue outline**, and widths in **red** are measured by the page after layout. The sizing maths is implemented in `code/flex_algorithm.mjs` and checked against headless Chrome by `code/03_flex_measure.mjs`.

---

## 8.1 Flex container and flex items

### What it is

`display: flex` makes an element a **flex container**; its **direct children** become **flex items**, laid out by flexbox instead of normal flow. Grandchildren are unaffected — each level decides for itself (so `display: flex` on a `<ul>` makes the `<li>`s items, not the `<a>`s inside them).

```
<nav class="toolbar">          <!-- flex container -->
  <a href="/">Home</a>         <!-- flex item -->
  <a href="/docs">Docs</a>     <!-- flex item -->
  <span>v2.1 <b>beta</b></span> <!-- flex item; the <b> inside is NOT an item -->
</nav>
```

```
.toolbar { display: flex; }          /* block-level container (takes a full line) */
.badge   { display: inline-flex; }   /* inline-level container (sits in text); also `inline flex` */
```

A bare run of text directly inside the container becomes an **anonymous flex item** (browser-created, no element, so not targetable by CSS).

### Which properties go where

| On the **container** | On the **items** |
|---|---|
| `display: flex \| inline-flex` | `flex-grow`, `flex-shrink`, `flex-basis` (or shorthand `flex`) |
| `flex-direction` | `align-self` |
| `flex-wrap` (both via `flex-flow`) | `order` |
| `justify-content` | `margin` (including `auto`) |
| `align-items` | `min-width` / `min-height` (matters a lot, see 8.4) |
| `align-content` | |
| `gap`, `row-gap`, `column-gap` | |

### What changes for the children

- Items are "blockified": `display: block` / `inline` no longer matters (a `<span>` item accepts width and height).
- `float` and `clear` are ignored; `vertical-align` has no effect.

- Margins of adjacent items **do not collapse** (§7.3).
- An item with `z-index` creates a stacking context even without `position` (§7.8).
- By default items sit in a row, are content-width, may shrink but not grow, and **stretch** to the container's height.

---

## 8.2 The two axes, `flex-direction` and `order`

### What it is

Flexbox never says "horizontal"/"vertical". It uses two **axes**:

- The **main axis** — the direction items are placed in, from **main-start** to **main-end**.
- The **cross axis** — perpendicular to it, **cross-start** / **cross-end**.

| `flex-direction` | Main axis (English, left-to-right text) | Cross axis |
|---|---|---|
| `row` (default) | left → right (follows text direction; right → left in Arabic or Hebrew) | top → bottom |
| `row-reverse` | right → left | top → bottom |
| `column` | top → bottom | left → right |
| `column-reverse` | bottom → top | left → right |

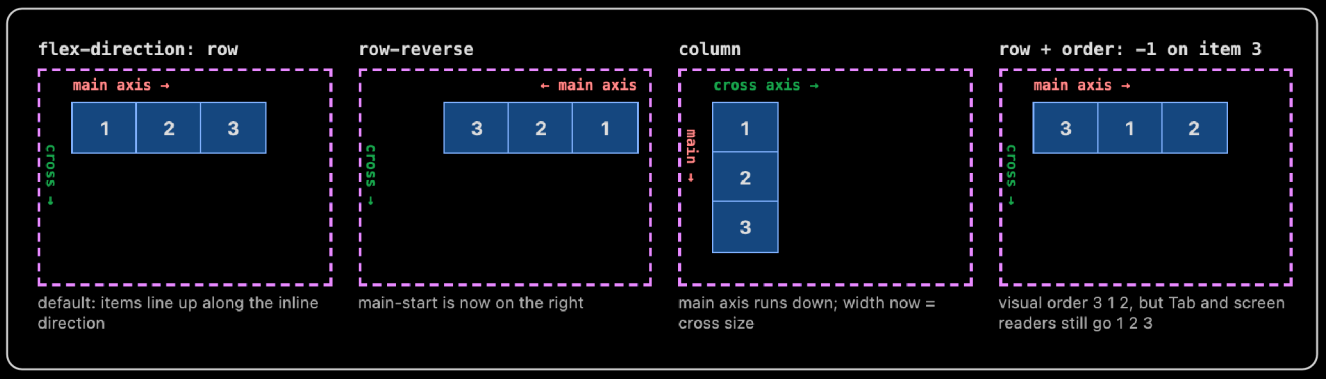



*Figure 8.1 — `code/01_axes.html`. Red arrow = main axis, green = cross axis (drawn with `::before` / `::after`).*

### Why it matters

**`justify-content` works on the main axis; `align-items`, `align-self` and `align-content` on the cross axis.** Switch to `flex-direction: column` and `justify-content: center` centres *vertically*, while `align-items: center` now centres horizontally. If you remember one thing about flexbox, remember this.

**`order`**

`order: <integer>` (default `0`) changes **visual** position: ascending `order`, ties in source order (values 2, −1, 0, 0 → the −1 first, then the 0s, then 2). In the fourth demo `order: -1` moves item 3 to the front.

### When not to use it (accessibility)

`order`, `row-reverse` and `column-reverse` change only what is *drawn*. **Tab order and screen-reader reading order still follow the DOM**, so keyboard focus jumps around — WCAG 2.2 SC 1.3.2 (Meaningful Sequence) and 2.4.3 (Focus Order) (Chapter 3).

- Change the HTML when the order means something; use `order` only for purely visual tweaks (an image before/after its text in alternating rows).
- The new CSS `reading-flow` property makes focus and reading order follow the visual flex/grid order, but as of 2026 only Chromium (Chrome/Edge 137+) ships it — not Baseline, so don't rely on it.

---

## 8.3 `flex-grow`, `flex-shrink` and `flex-basis`

### What it is

Three sizing properties, read **along the main axis** (width in a row, height in a column):

| Property | Meaning | Default |
|---|---|---|
| `flex-basis` | Starting size before free space is shared: a length (`200px`, `30%`), `0`, `content`, or `auto` (= `width`/`height` if set, else content size) | `auto` |
| `flex-grow` | How many *shares* of **positive free space** the item takes | `0` (don't grow) |
| `flex-shrink` | How strongly the item gives up space when there is **negative free space** (overflow), weighted by its basis | `1` (may shrink) |

**Free space** = container's inner main size − Σ items' flex base sizes (plus margins, borders, padding) − gaps. Positive → grow; negative → shrink. (Like splitting a bill: leftover budget is shared by grow "shares"; an overspend is paid back in proportion to shrink × what each ordered.)

### How it works — the algorithm, simplified

"Resolving the flexible lengths" (Flexbox Level 1, §9.7), for one line:

1. **Base sizes.** Each item's flex base size from `flex-basis`, clamped by `min-*`/`max-*` → its *hypothetical* size.
2. **Grow or shrink?** If Σ hypothetical sizes + gaps + margins/padding/borders < container, use `flex-grow`; otherwise `flex-shrink`.
3. **Freeze inflexible items** (factor 0) at their hypothetical size ("frozen" = final).
4. **Distribute** remaining free space among unfrozen items:
   - growing: `size = basis + free × grow / Σgrow`
   - shrinking: `size = basis − overflow × (shrink × basis) / Σ(shrink × basis)`
   - if unfrozen factors sum to **less than 1**, only that fraction of the free space is handed out.
5. **Clamp** each to `min-*`/`max-*`; if anything was clamped, **freeze** it there and redo step 4 with the remaining space.
6. Stop when every item is frozen.

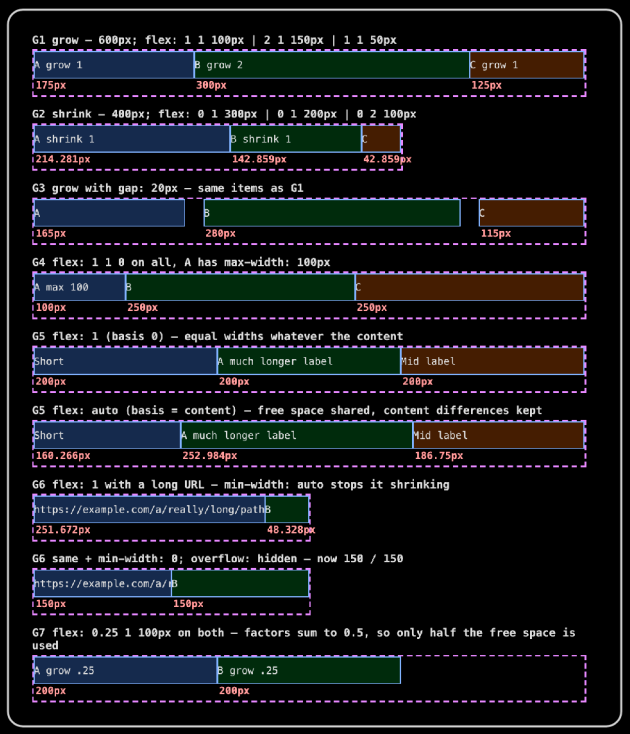


*Figure 8.2 — `code/02_grow_shrink.html`. Items have no padding/border, so width = flex size. A script writes each `getBoundingClientRect()` width underneath (labels are absolutely positioned, so they don't affect the layout they report).*

### Worked example 1 — growing (G1)

Container 600px; items `flex: 1 1 100px`, `flex: 2 1 150px`, `flex: 1 1 50px`.

1. Bases 300 → free **+300** → grow.
2. Shares 1 + 2 + 1 = 4 → one share = 75px.
3. A = 100 + 75 = **175**; B = 150 + 150 = **300**; C = 50 + 75 = **125**. ✓ 600.

B is **not** twice A — `flex-grow` divides the *free space*, not final widths; grow factors become width ratios only when all bases are 0.

### Worked example 2 — shrinking (G2)

Container 400px; items `flex: 0 1 300px`, `flex: 0 1 200px`, `flex: 0 2 100px`.

1. Bases 600 → free **−200** → shrink.
2. Scaled factors (shrink × basis): 300, 200, 200 → Σ **700**.
3. A loses 200 × 300/700 = 85.714 → **214.286**; B loses 57.143 → **142.857**; C loses 57.143 → **42.857**.

C has *twice* B's shrink factor yet loses the *same* amount, because its basis is half — weighting by size stops small items being crushed first.

### Worked example 3 — gaps (G3)

G1 with `gap: 20px`: two gaps take 40px first → free 260 → share 65 → **165 / 280 / 115**.

### Worked example 4 — a max-width freezes an item (G4)

Container 600px, all `flex: 1 1 0`, A has `max-width: 100px`.

1. Round 1: each gets 200; A violates its max → clamped to 100, **frozen**.
2. Round 2: 500 left for B and C → **250** each.

Result **100 / 250 / 250**, not 100/200/200 with 100px wasted — that's the step-5 loop.

### Worked example 5 — factors below 1 (G7)

Container 600px, two items `flex: 0.25 1 100px`: free 400, factors sum 0.5 → only 200 handed out → **200 / 200**, 200px left empty (with `flex-grow: 1` each: 300 / 300).

### Verified — the maths vs. Chrome

`code/02_flex_math.mjs` runs `flex_algorithm.mjs` with a per-round log (the core is the two distribution formulas above, then clamp-freeze-repeat). `code/03_flex_measure.mjs` then reads every item's `getBoundingClientRect().width` in headless Chrome and compares; for G5 and G6 it first measures content sizes — **min-content** (narrowest without overflow, roughly the longest word) and **max-content** (width with no line breaks) — on a hidden clone:

```
case   predicted (spec algorithm)          measured in Chrome                 diff
g1     175 / 300 / 125                     175 / 300 / 125                    0
g2     214.286 / 142.857 / 42.857          214.281 / 142.859 / 42.859         0.005
g3     165 / 280 / 115                     165 / 280 / 115                    0
g4     100 / 250 / 250                     100 / 250 / 250                    0
g5a    200 / 200 / 200                     200 / 200 / 200                    0
g5b    160.266 / 252.984 / 186.75          160.266 / 252.984 / 186.75         0
g6a    251.672 / 48.328                    251.672 / 48.328                   0
g6b    150 / 150                           150 / 150                          0
g7     200 / 200                           200 / 200                          0

content sizes used above (min-content / max-content):
  g5b 33.125 / 33.125  |  125.844 / 125.844  |  59.609 / 59.609
  g6a 251.672 / 251.672  |  6.625 / 6.625
```

Every case matches. The tiny G2 difference: Chrome stores layout positions in **1/64-pixel units** (214.2857… → 13714/64 = 214.28125), pushing rounding leftovers onto neighbours so the three still sum to exactly 400.

### Common mistakes

- Expecting `flex-grow: 2` to be twice as wide as a `flex-grow: 1` sibling — true only with equal (typically 0) bases.
- Forgetting padding and border count toward the base: a `flex: 1` item with `padding: 0 40px` ends up 80px wider than an unpadded sibling.
- `flex-shrink: 0` on everything, then a row overflowing on phones.

### Self-check

1. Container 500px, items `flex: 1 1 100px` and `flex: 3 1 100px`? → free 300, 4 shares of 75 → **175 and 325**.
2. Container 300px, items `flex: 0 1 200px` and `flex: 0 3 200px`? → scaled 200 and 600 (Σ 800); losses 25 and 75 → **175 and 125**.

---

## 8.4 The `flex` shorthand and minimum sizes

### The shorthand

Prefer `flex` over the three longhands: it sets them together with sensible defaults for omitted parts (omitted `flex-shrink` → 1, omitted `flex-basis` → **0%**, not `auto`).

| You write | Means (grow shrink basis) | Behaviour | Typical use |
|---|---|---|---|
| *(nothing)* / `flex: initial` | `0 1 auto` | Content-sized, may shrink, won't grow | Default |
| `flex: 1` | `1 1 0%` | Ignore content size, share **all** space equally | Equal columns, "fill the rest" |
| `flex: 2` | `2 1 0%` | Twice the share of a `flex: 1` sibling | Proportional columns |
| `flex: auto` | `1 1 auto` | Start from content size, share the **leftover** equally | Nav tabs that grow but keep content differences |
| `flex: none` | `0 0 auto` | Rigid: never grows or shrinks | Icons, avatars, buttons |
| `flex: 200px` | `1 1 200px` | Start at 200px, grow/shrink from there | Wrapping card rows |
| `flex: 0 0 200px` | exactly that | Fixed 200px | Sidebars |

**G5** shows `flex: 1` vs `flex: auto` with the same three labels: `flex: 1` gives 200/200/200 regardless of text; `flex: auto` starts from the text widths (33.1, 125.8, 59.6) and adds an equal 127.1px each → 160.3 / 253.0 / 186.8.

### `min-width: auto` — the hidden minimum

Flex items have `min-width: auto` (`min-height: auto` in a column), which for flex items means **"never shrink below my min-content size"** — the longest word, URL or unshrinkable child. It protects text, but causes the most common flexbox bug: *"my `flex: 1` item overflows its container"*.

**G6**: two `flex: 1` items in 300px should be 150/150, but the first contains an unbreakable URL with min-content 251.672px, so the second gets only 48.328px. The fix:

```
.item { flex: 1; min-width: 0; }            /* allow shrinking below content size */
/* plus a way to deal with the overflowing content: */
.item { overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }  /* or overflow-wrap: anywhere */
```

Where it bites: a code block or table in a flex column, a long file name in a list row, a chart in a flex child, nested flex layouts with a wide `<pre>` (fix: `min-width: 0` on the item, `overflow: auto` on the `<pre>`). Making the item a scroll container (`overflow: hidden/auto/scroll`) also removes the automatic minimum; `overflow: clip` doesn't (not a scroll container).

---

## 8.5 Alignment

### What it is

After sizing there may still be **free space** — on the main axis (items don't fill the line) and the cross axis (items shorter than the line). Alignment properties (from the CSS Box Alignment module, shared with Grid) decide where it goes.

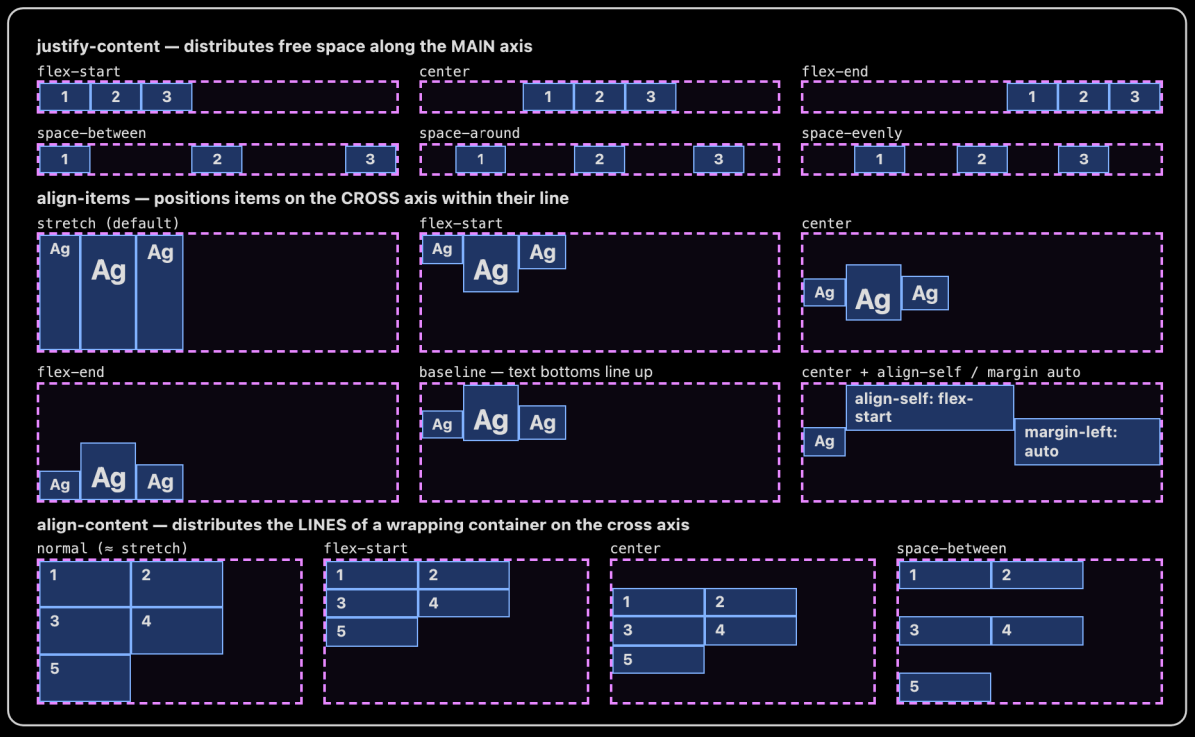



*Figure 8.3 — `code/04_alignment.html`.*

### `justify-content` — main axis, on the container

| Value | Where the free space goes |
|---|---|
| `flex-start` (effectively the default, `normal`) | All at the end; items packed at main-start |
| `flex-end` | All at the start |
| `center` | Half on each side |
| `space-between` | Between items only; first and last touch the edges |
| `space-around` | Equal space around each item, so edges get **half** a gap |
| `space-evenly` | Equal gaps everywhere, edges included |

`start` / `end` follow the writing direction rather than the flex direction (matters only with `*-reverse`). `justify-content` does nothing if an item has `flex-grow` > 0 with no max-size — no free space remains.

### `align-items` and `align-self` — cross axis, per line

| Value | Effect on each item |
|---|---|
| `stretch` (default — `normal` behaves like it) | Items fill the line's cross size — **only if** their cross size (`height` in a row) is `auto` (a `height: 50px` item stays 50px) |
| `flex-start` / `start` | Top of the line (in a row) |
| `flex-end` / `end` | Bottom |
| `center` | Middle |
| `baseline` | Items shifted so each one's **first line of text** shares a baseline — for a label beside a big number or mixed font sizes |

`align-items` sets the default; `align-self` overrides it per item. There is **no** `justify-self` in flexbox — the main axis is shared — use an **auto margin**.

#### Auto margins

`margin: auto` absorbs free space *before* `justify-content`, in either axis:

- `margin-left: auto` (or `margin-inline-start: auto`) pushes the item — and everything after — to the end ("logo and links left, Sign in far right").
- `margin-top: auto` on a column's last item pushes it to the bottom (a card's button).
- `margin: auto` on an only child centres it both ways.

#### `align-content` — distributing the lines

With `flex-wrap: wrap` there can be several **flex lines**; `align-content` distributes them on the cross axis (`flex-start`, `center`, `space-between`, `space-around`, `space-evenly`, `stretch`). Default `normal` behaves as `stretch`. **No effect on a single-line (`nowrap`) container.**

#### Common mistakes

- `align-items: center` to centre horizontally in a row (that's `justify-content`).
- A `height` on items, then wondering why `stretch` stopped.
- `align-content` on a non-wrapping container.
- `justify-content: center` on an overflowing row: content spills off *both* sides and the left part can't be scrolled to. `justify-content: safe center` falls back to start on overflow (Baseline since 2024 — Chrome 115, Firefox 63, Safari 17.6); or `margin: auto` on items, which never pushes content past the start.

---

## 8.6 Wrapping and `gap`

**`flex-wrap`**

| Value | Behaviour |
|---|---|
| `nowrap` (default) | One line. Items shrink (if allowed), then **overflow** |
| `wrap` | New lines when the next item doesn't fit; lines stack along the cross axis |
| `wrap-reverse` | Same, but new lines go *above* (cross-start and cross-end swap) |

`flex-flow: row wrap` = `flex-direction` + `flex-wrap`. Each line is sized **independently**: items are placed by *hypothetical* size (basis clamped by min/max) — as many as fit — then the 8.3 maths runs per line.

**`gap`**

`gap: <row-gap> <column-gap>` (or one value) puts fixed space **between** items and lines — never on the outer edges. Not a margin, never collapses, subtracted before free space is shared (G3). Baseline for flexbox

since 2021 (before: item margins plus a negative container margin).

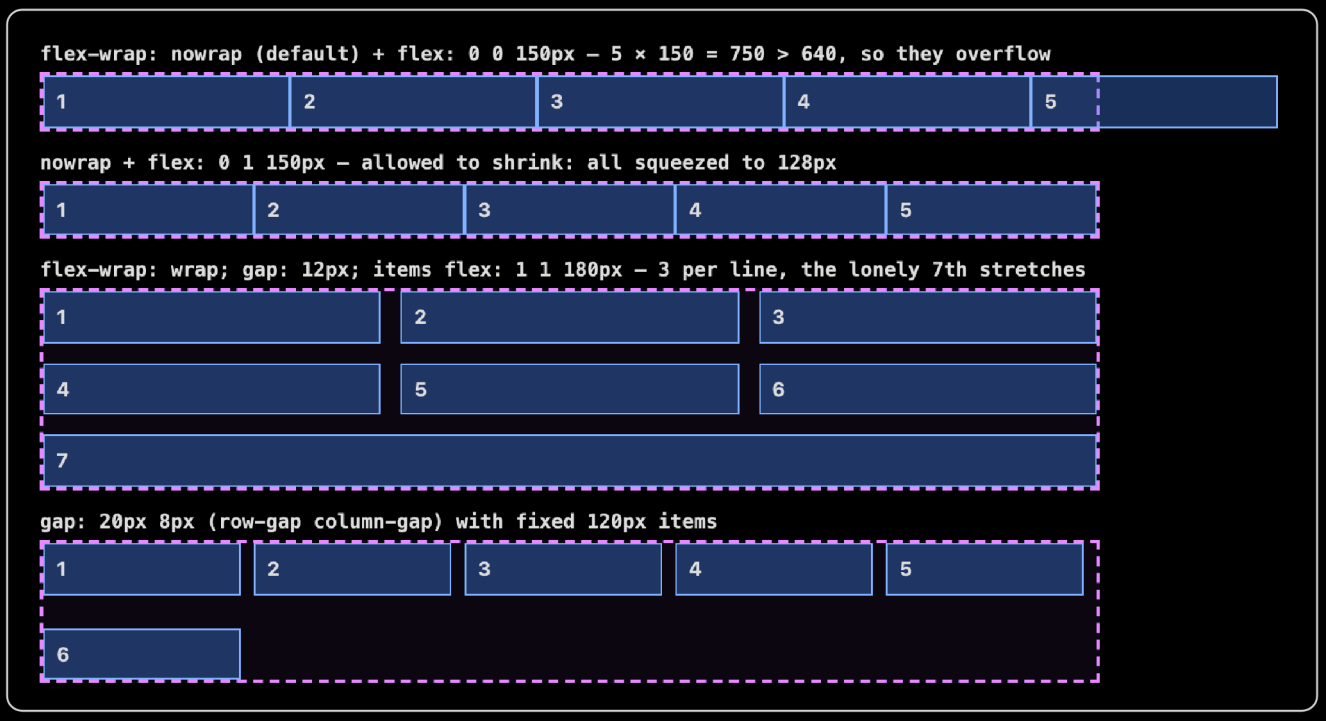



*Figure 8.4 — `code/05_wrap_gap.html`.*

`code/05_wrap_gap.mjs` groups cards by top edge to count items per line:

```
container 640px (flex: 1 1 180px, gap 12px):
  line 1: 3 item(s), widths 205.33, 205.34, 205.33
  line 2: 3 item(s), widths 205.33, 205.34, 205.33
  line 3: 1 item(s), widths 640
container 400px (flex: 1 1 180px, gap 12px):
  line 1: 2 item(s), widths 194, 194
  line 2: 2 item(s), widths 194, 194
  line 3: 2 item(s), widths 194, 194
  line 4: 1 item(s), widths 400
nowrap + flex: 0 1 150px in 640px: 128, 128, 128, 128, 128
```

By hand: 3 × 180 + 2 × 12 = 564 ≤ 640, but 4 × 180 + 3 × 12 = 756 > 640, so 3 fit; free 76 → +25.33 each → 205.33. The lone last card takes all 640px. Shrink row: overflow 750 − 640 = 110, equal weights → each loses 22 → 128.

### The "last row" problem — and when to switch to grid

Wrapping flex is **content-out** (content drives sizes): each line shares its own free space, so items on a short last line grow wider than those above. Great for tags or buttons of different lengths; wrong for a uniform card grid. For equal wrapping columns use Grid's `repeat(auto-fill, minmax(180px, 1fr))` (Chapter 9), which is **layout-in**: tracks are defined first and items drop into them. (Flex workaround: `flex-grow: 0` with a fixed basis, at the cost of an uneven right edge.)

---

## 8.7 Common patterns

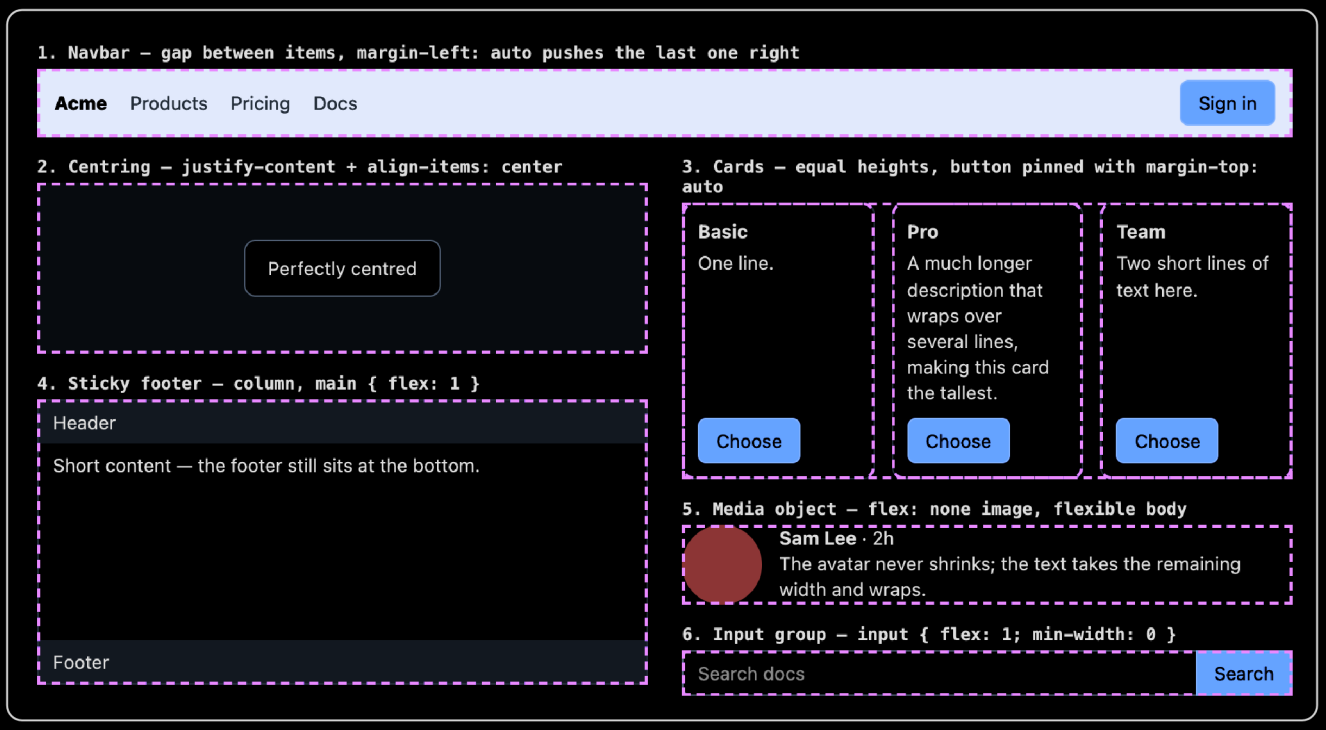



*Figure 8.5 — `code/06_patterns.html`.*

```css
/* 1. Navbar */
.nav       { display: flex; align-items: center; gap: 16px; }
.nav .push { margin-left: auto; }                    /* everything from here on goes right */

/* 2. Centre anything, both axes */
.center { display: flex; justify-content: center; align-items: center; }
/* (grid alternative: display: grid; place-items: center) */

/* 3. Equal-height cards with buttons aligned at the bottom */
.cards       { display: flex; gap: 12px; }            /* align-items: stretch -> equal heights */
.card        { flex: 1; display: flex; flex-direction: column; }
.card button { margin-top: auto; align-self: flex-start; }

/* 4. Sticky footer: footer at the bottom even when content is short */
body      { display: flex; flex-direction: column; min-height: 100dvh; }   /* demo uses 200px */
body main { flex: 1; }

/* 5. Media object: fixed image, flexible text */
.media        { display: flex; gap: 12px; align-items: flex-start; }
.media .avatar { flex: none; width: 56px; height: 56px; }
.media .body   { flex: 1; min-width: 0; }

/* 6. Input group */
.group       { display: flex; }
.group input { flex: 1; min-width: 0; }
```

Measured by `code/06_patterns.mjs`:

```
navbar : gap nav-right-edge minus button-right-edge = 12px (the 12px padding)
centre : space left/right 144.65625 / 144.65625, top/bottom 39.90625 / 39.90625
cards  : heights 193.5, 193.5, 193.5; button bottoms 308.6875, 308.6875, 308.6875
footer : page bottom 464.1875, footer bottom 463.1875 (only the 1px border below it)
media  : avatar width 56px (flex: none, never shrinks)
input  : input 359.875px + button 68.125px = form 428px
```

- **Navbar**: in production a real `<nav>` with a list; the `<ul>` is the flex container (Chapter 3). Add `flex-wrap: wrap` or a menu button for narrow screens (Chapter 10).

- **Centring**: works for unknown sizes on both axes. If the child can be taller than the container, prefer `margin: auto` on it so it can't overflow off the top.
- **Cards**: equal heights from `align-items: stretch`; column + `margin-top: auto` aligns buttons. Aligning *titles and paragraphs* across cards needs `subgrid` (Chapter 9).
- **Sticky footer** (footer at the bottom of short pages): `100dvh` = *dynamic* viewport height, accounting for mobile toolbars; `100vh` can be too tall on phones (Chapter 10).
- **Media object** (image beside text): `flex: none` keeps the avatar unsquashed; `min-width: 0` lets long names truncate.
- **Input group** (text box joined to a button): `<input>` has a default width from its `size` attribute, so without `min-width: 0` it may refuse to shrink.

### Real-world examples

GitHub's repository header, most navbars, Bootstrap 5's `.d-flex` / `.justify-content-between` and Tailwind's `flex`, `items-center`, `justify-between`, `gap-4` (utility classes, one property each) map one-to-one to this chapter's properties.

---

## 8.8 Pitfalls, debugging, and interview prep

### Pitfalls at a glance

| Symptom | Cause | Fix |
|---|---|---|
| Item with `flex: 1` overflows / text doesn't truncate | `min-width: auto` = min-content | `min-width: 0` (and `overflow` / `text-overflow` on the content) |
| Image in a flex row is squashed | Images shrink like any item | `flex: none` or `flex-shrink: 0` |
| Image in a row becomes tall and distorted or overflows | `align-items: stretch` stretches it to the line (a 100×50 image in a 300px-tall row became 600×300 in Chrome) | `align-self: flex-start` (or `center`) |
| `align-items: stretch` has no effect | Item has an explicit `height` | Remove it or use `min-height` |
| `height: 100%` on an item's child doesn't work | Parent's height isn't definite (fixed in advance) | Make the item a flex container too, or `flex: 1` down the chain |
| `justify-content` does nothing | An item has `flex-grow`, so no free space remains | Remove the grow, or use `max-width` |
| Last row of wrapped cards is wider | Per-line sizing | Use grid |
| Tab order doesn't match what users see | `order` / `*-reverse` | Change the HTML instead |
| Unexpected space between items | `justify-content: space-*` or leftover `margin` | Inspect in DevTools |

### Debugging with DevTools

Chrome, Edge and Firefox show a **flex badge** beside flex containers in the Elements panel (F12); clicking it overlays container, items, free space and gaps. The **Flexbox editor** (icon by `display: flex` in Styles) clicks through `justify-content` / `align-items` values. Firefox's Layout panel explains *why* an item has its size: base size, growth/shrink, and any min/max clamp.

### In the interview

> **30-second answer:** "Flexbox is one-dimensional layout. `display: flex` makes direct children flex items, placed along a main axis set by `flex-direction`; the cross axis is perpendicular. `justify-content` distributes space on the main axis, `align-items` / `align-self` align on the cross axis, and `align-content` distributes wrapped lines. Sizes start from `flex-basis`; positive free space is shared by `flex-grow` ratios and overflow taken back in proportion to `flex-shrink` × basis, with min/max clamps. `flex: 1` means `1 1 0%`, so items share all the space equally. The classic bug is `min-width: auto`, which stops items shrinking below their content — fix with `min-width: 0`."

- *`flex: 1` vs `flex: auto`?* → Basis 0% vs auto: `flex: 1` ignores content size and splits everything; `flex: auto` keeps content differences and splits only the leftover (G5).
- *How is shrinking calculated?* → Overflow × (shrink × basis) / Σ(shrink × basis) — weighted by basis so small items don't vanish (G2: 214.29 / 142.86 / 42.86).
- *Flexbox or grid for a page layout?* → Grid for 2-D page structure and uniform card grids; flexbox inside components for content-sized rows. Often nested.
- *How do you centre a div?* → `display: flex; justify-content: center; align-items: center` on the parent, or `display: grid; place-items: center`.

---

## Chapter 8 TL;DR

- `display: flex` → direct children become **flex items**; no margin collapsing, floats ignored, items blockified.
- **Main axis** = `flex-direction` (`row` default). `justify-content` → main axis. `align-items` / `align-self` / `align-content` → cross axis. Swap direction and the meanings swap.
- `order` and `*-reverse` change only visuals, not Tab or reading order (`reading-flow` is Chromium-only, not Baseline).
- **Sizing**: start from `flex-basis`; positive free space split by `flex-grow` shares (`basis + free × grow/Σgrow`); overflow taken back by `flex-shrink × basis` weights; clamp to min/max, freeze, repeat. Verified: 175/300/125 (grow), 214.29/142.86/42.86 (shrink), 100/250/250 (max-width freeze). Chrome stores sizes in 1/64 px.
- **Shorthand**: `flex: 1` = `1 1 0%`, `auto` = `1 1 auto`, `none` = `0 0 auto`, default `0 1 auto`, `flex: 200px` = `1 1 200px`.
- **`min-width: auto`** = min-content for flex items → overflowing `flex: 1` items; fix with `min-width: 0`.
- **Auto margins** absorb free space: `margin-left: auto` pushes right, `margin-top: auto` pushes down, `margin: auto` centres.
- **Wrap + gap**: lines are sized independently; `gap` only between items; for an even last row use grid.
- **Patterns**: navbar, centring, equal-height cards with bottom-aligned buttons, sticky footer (`min-height: 100dvh` + `main { flex: 1 }`), media object, input group.

Next: Chapter 9 — CSS Grid.